

Part I: Code

a. Kernel K Means :

-> **Main function workflow:**

Parameters taken `GAMMA_S = 0.001 , GAMMA_C = 0.001, NUM_CLUSTERS = 2`
`MAX_ITER = 100`

1. The program first reads the input image and extracts the pixel coordinates and RGB color values.
2. A kernel matrix is then constructed using both spatial and color information.
3. After initializing the clusters using K-Means++, Kernel K-Means is performed iteratively until convergence.
4. The segmentation result of each iteration is saved as an image, and all frames are combined into a GIF at the end

```
for i in IMAGE_PATH:
    img = np.array(Image.open(i))
    h, w, _ = img.shape
    coords = np.array([[i, j] for i in range(h) for j in range(w)])
    colors = img.reshape(-1, 3)
    print("Building kernel matrix...")
    kernel = build_kernel(coords, colors, GAMMA_S, GAMMA_C)
    print("Initializing clusters...")
    labels = initialize_clusters(coords, colors, NUM_CLUSTERS)

    print("Running Kernel K-Means...")
    final_labels, frame_files = kernel_kmeans(kernel, labels, NUM_CLUSTERS, (h, w), MAX_ITER)
    print("Creating GIF...")
    create_gif(frame_files, os.path.join(OUTPUT_DIR, f"kernel_kmeans{i}.gif"))
    print("Finished.")
```

-> **Reading images and extracting value:** As per the assignment requirement the kernel used is below function:

$$k(x, x') = e^{-\gamma_s \|S(x) - S(x')\|^2} \times e^{-\gamma_c \|C(x) - C(x')\|^2}$$

Where: $S(x)$ denotes the spatial coordinate (row,col) of pixel x

- $C(x)$ denotes the RGB color vector of pixel x
- γ_s controls the influence of spatial distance
- γ_c controls the influence of color distance

The squared Euclidean distances between all pixel coordinates and all pixel colors are first computed using the `cdist` function. The Gaussian similarity for spatial information and color information is then calculated and multiplied together to obtain the final kernel matrix.

```
def build_kernel(coords, colors, gamma_s, gamma_c):
    spatial_dist2 = cdist(coords, coords, metric="sqeuclidean")
    color_dist2 = cdist(colors, colors, metric="sqeuclidean")

    kernel = (np.exp(-gamma_s * spatial_dist2) * np.exp(-gamma_c * color_dist2))
    return kernel.astype(np.float32)
```

->Computing the distance:

`compute_distance()` Kernel K-Means cannot directly compute centroids in kernel space.

Therefore, distances between pixels and clusters are computed using the kernel matrix.

$$d(x_i, C_k)^2 = K(x_i, x_i) - \frac{2}{|C_k|} \sum_{x_j \in C_k} K(x_i, x_j) + \frac{1}{|C_k|^2} \sum_{x_p, x_q \in C_k} K(x_p, x_q)$$

Here : $|C_k|$ is the number of pixels in cluster C_k .

$K(x_i, x_i)$ is the self-similarity of pixel (x_i, x_i)

```
def compute_distance(kernel, kernel_diag, labels, cluster_id):
    cluster = np.where(labels == cluster_id)[0]
    nk = len(cluster)
    if nk == 0:
        return np.full(kernel.shape[0], np.inf)
    # K(x_i, x_i)
    term1 = kernel_diag
    # (2 / |C_k|) * sum K(x_i, x_j)
    term2 = (2.0 / nk) * np.sum(kernel[:, cluster], axis=1)
    # (1 / |C_k|^2) * sum K(x_p, x_q)
    term3 = (1.0 / (nk * nk)) * np.sum(kernel[np.ix_(cluster, cluster)])
    return term1 - term2 + term3
```

->Initialising the clusters:

This function performs K-Means++ initialization to obtain better starting clusters than random initialization.

Spatial coordinates and color values are first normalized and combined into a 5-dimensional feature vector:

$$X_i = (\text{row}_i, \text{col}_i, R_i, G_i, B_i)$$

K-Means++ Procedure:

1. It selects the first center randomly. Later we compute the squared distance from every point to its nearest selected center.
2. We select the next center with probability proportional to that distance. Repeat until k centers are obtained.

```
def initialize_clusters(coords, colors, n_clusters):
    coords_norm = coords / 99.0
    colors_norm = colors / 255.0
    features = np.hstack([coords_norm, colors_norm])
    n = features.shape[0]
    centers = []
    first_center = np.random.randint(n)
    centers.append(first_center)
    for _ in range(n_clusters - 1):
        min_dist = np.full(n, np.inf)
        for c in centers:
            dist = np.sum((features - features[c]) ** 2, axis=1)
            min_dist = np.minimum(min_dist, dist)
        total = np.sum(min_dist)
        if total == 0:
            next_center = np.random.randint(n)
        else:
            prob = min_dist / total
            next_center = np.random.choice(n, p=prob)
        centers.append(next_center)
    centers = np.array(centers)
    distances = []
    for c in centers:
        d = np.sum((features - features[c]) ** 2, axis=1)
        distances.append(d)
    distances = np.array(distances)
    labels = np.argmin(distances, axis=0)
    return labels
```

-> Iteratively computing the K mean clusters:

kernel_kmeans() This function performs the iterative Kernel K-Means clustering process.

First it computes kernel distances between every pixel and every cluster.

$$d(X_i, C_1) \dots d(X_i, C_2) \dots d(X_i, C_k)$$

Later we assign each pixel to the nearest cluster.

$$\text{Label}(X_i) = \arg \min(k) d(X_i, C_k)$$

If: labels=new_labels → then no pixels changed clusters and the algorithm terminates.

Otherwise, the process repeats until convergence or the maximum number of iterations is reached.

Post computation we return the labels assigned and the list of files generated in the process.

```
def kernel_kmeans(kernel, labels, n_clusters, image_shape, max_iter=100):
    kernel_diag = np.diag(kernel)
    frame_files = []
    first_frame = os.path.join(OUTPUT_DIR, "frame_000.png")
    save_segmentation(labels, image_shape, first_frame)
    frame_files.append(first_frame)
    for iteration in range(max_iter):
        distances = []
        for k in range(n_clusters):
            d = compute_distance(kernel, kernel_diag, labels, k)
            distances.append(d)
        distances = np.array(distances)
        new_labels = np.argmin(distances, axis=0)
        filename = os.path.join(OUTPUT_DIR, f"frame_{iteration+1:03d}.png")
        save_segmentation(new_labels, image_shape, filename)
        frame_files.append(filename)
        changed = np.sum(labels != new_labels)
        print(f"Iteration {iteration+1}: {f'{changed}' pixels changed}")
        if np.array_equal(labels, new_labels):
            print("Converged")
            break
        labels = new_labels
    return labels, frame_files
```

Saving the Segments and creating Gifs: This function converts cluster labels into a color image for visualization.

Each cluster is assigned a predefined RGB color.

```
def save_segmentation(labels, image_shape, filename):
    cluster_colors = np.array([[255, 0, 0], [0, 255, 0], [0, 0, 255], [255, 255, 0], [255, 0, 255], [0, 255, 255]])
    rgb = cluster_colors[labels % len(cluster_colors)]
    rgb = rgb.reshape(image_shape[0], image_shape[1], 3)
    img = Image.fromarray(rgb.astype(np.uint8))
    img.save(filename)
```

This create_gif() function visualizes the clustering process by generating an animated GIF.

1. Load every segmentation image generated during the iterations.
2. Store the images as frames.
3. Combine the frames into a GIF.

The resulting animation shows how cluster assignments evolve until convergence.

```
def create_gif(frame_files, output_name):
    gif_dir = os.path.join(os.path.dirname(output_name), "gif")
    os.makedirs(gif_dir, exist_ok=True)
    output_path = os.path.join(gif_dir, os.path.basename(output_name))
    frames = [Image.open(file) for file in frame_files]
    frames[0].save(output_path, format="GIF", append_images=frames[1:], save_all=True, duration=500, loop=2)
```

b. Spectral Clustering:

-> Main function workflow:

Parameters taken

```
GAMMA_C = 0.001
NUM_CLUSTERS = 2
MAX_ITER = 100
MODE = "ratio"          # "ratio" or "normalized"
OUTPUT_DIR = f"spectral_{MODE}_output"
```

1. The program first reads the input image and extracts the pixel coordinates and RGB color values.
2. A kernel matrix is then constructed using both spatial and color information.
3. After initializing the clusters using K-Means++, Kernel K-Means is performed iteratively until convergence.
4. The segmentation result of each iteration is saved as an image, and all frames are combined into a GIF at the end

```
img = np.array(Image.open(IMAGE_PATH))
h, w, _ = img.shape
coords = np.array([[i, j] for i in range(h) for j in range(w)], dtype=np.float32)
colors = img.reshape(-1, 3).astype(np.float32)
print("Building kernel...")
W = build_kernel(coords, colors, GAMMA_S, GAMMA_C)
print("Building Laplacian...")
if MODE == "ratio":
    L = ratio_cut_laplacian(W)
elif MODE == "normalized":
    L = normalized_cut_laplacian(W)
else:
    raise ValueError("MODE must be ratio or normalized")

del W
U = compute_eigenvectors(L, NUM_CLUSTERS)
del L
if MODE == "normalized":
    U = normalize_rows(U)
print("Initializing clusters...")
labels = initialize_clusters(U, NUM_CLUSTERS)
print("Cluster sizes:", np.bincount(labels))

print("Running spectral clustering...")
final_labels, frame_files = kmeans(U, labels, NUM_CLUSTERS, (h, w), MAX_ITER)
print("Creating GIF...")
create_gif(frame_files, os.path.join(OUTPUT_DIR, f"spectral_{MODE}.gif"))
print("Finished.")
```

-> Reading images and extracting value: As per the assignment requirement the kernel used is below function:

$$k(x, x') = e^{-\gamma_s \|S(x) - S(x')\|^2} \times e^{-\gamma_c \|C(x) - C(x')\|^2}$$

Where: $S(x)$ denotes the spatial coordinate (row,col) of pixel x

- $C(x)$ denotes the RGB color vector of pixel x
- γ_s controls the influence of spatial distance
- γ_c controls the influence of color distance

The squared Euclidean distances between all pixel coordinates and all pixel colors are first computed using the `cdist` function. The Gaussian similarity for spatial information and color information is then calculated and multiplied together to obtain the final kernel matrix.

```
def build_kernel(coords, colors, gamma_s, gamma_c):
    spatial_dist2 = cdist(coords, coords, metric="sqeuclidean").astype(np.float32)
    color_dist2 = cdist(colors, colors, metric="sqeuclidean").astype(np.float32)
    kernel = np.exp(-gamma_s * spatial_dist2, dtype=np.float32)
    del spatial_dist2
    kernel *= np.exp(-gamma_c * color_dist2, dtype=np.float32)
    del color_dist2
    return kernel
```

->**Ratio cut laplasian:** `ratio_cut_laplacian()` function Constructs the unnormalized graph Laplacian used for Ratio Cut spectral clustering.

The degree of each node is $d_i = \sum_j W_{ij}$

The graph Laplacian is computed as: $L = D - W$

```
def ratio_cut_laplacian(W):
    degree = np.sum(W, axis=1)
    L = -W.copy()
    np.fill_diagonal(L, degree)
    return L
```

->**Normalised cut laplasian:** `normalised_cut_laplacian()` function Constructs the normalized graph Laplacian used for Normalised Cut spectral clustering.

The degree of each node is $d_{ii} = \sum_j W_{ij}$

The graph Laplacian is computed as: $L = D - W$

```
def normalized_cut_laplacian(W):
    degree = np.sum(W, axis=1)
    D_inv_sqrt = 1.0 / np.sqrt(degree + 1e-12)
    L = -W.copy()
    np.fill_diagonal(L, degree)
    L *= D_inv_sqrt[:, None]
    L *= D_inv_sqrt[None, :]
    return L
```

Symmetric Normalized Laplacian-> The normalized Laplacian is:

$$L(\text{sym}) = 1/\sqrt{D} * L * 1/\sqrt{D}$$

This normalization compensates for large differences in node connectivity and often produces more balanced clusters.

Difference Between Ratio Cut and Normalized Cut

Ratio Cut

Uses the unnormalized Laplacian:

$$L = D - W$$

and directly performs eigenvalue decomposition on L.

Normalized Cut

Uses the normalized Laplacian:

$$L(\text{sym}) = 1/\sqrt{D} * L * 1/\sqrt{D}$$

followed by row normalization:

$$t_i = u_i / \|u_i\|$$

before applying K-Means.

The normalization reduces the influence of highly connected nodes and generally produces more balanced partitions.

-> **Calculating the Eigenvectors:** Performs eigenvalue decomposition of the Laplacian matrix.

Eigenvalue Problem

Spectral clustering solves:

$$Lv = \lambda v ; \text{ where } v = \text{eigenVector and } \lambda \text{ is the eigenvalue}$$

where:

- v is an eigenvector
- λ is an eigenvalue

Since the smallest eigenvalue is always approximately zero, its eigenvector is discarded.

The next k eigenvectors are selected:

$$U = [v_2, v_3, \dots, v_{k+1}]$$

For $k=2$: $\rightarrow U = [v_2, v_3]$

Each row of U becomes the new representation of a pixel in spectral space.

```
def compute_eigenvectors(L, n_clusters):  
    print("Computing eigenvectors...")  
    eigenvalues, eigenvectors = np.linalg.eigh(L)  
    U = eigenvectors[:,1:n_clusters+1].copy()  
    del eigenvectors  
    del eigenvalues  
    return U.astype(np.float32)
```

—> **Initialising the clusters:**

After all centers are obtained, each pixel is assigned to its nearest center.

$$\text{Label} (u_i) = \arg \min_c \|u_i - c\|^2$$

The resulting labels serve as the starting point for K-Means.

```
def initialize_clusters(features, n_clusters):
    n = features.shape[0]
    centers = []
    first_center = np.random.randint(n)
    centers.append(first_center)
    for _ in range(n_clusters - 1):
        min_dist = np.full(n, np.inf)
        for c in centers:
            dist = np.sum((features - features[c])**2, axis=1)
            min_dist = np.minimum(min_dist, dist)
        total = np.sum(min_dist)
        if total == 0:
            next_center = np.random.randint(n)
        else:
            prob = min_dist / total
            next_center = np.random.choice(n, p=prob)
        centers.append(next_center)
    centers = np.array(centers, dtype=np.int32)
    distances = []
    for c in centers:
        d = np.sum((features - features[c])**2, axis=1)
        distances.append(d)
    distances = np.array(distances)
    labels = np.argmin(distances, axis=0)
    return labels
```

—> **K-Means Clustering:** After cluster initialisation, the code for k means iteratively refines cluster assignments in the spectral embedding space until convergence. The objective is to minimize the within-cluster variance.

Mstep: For each cluster C_k , the centroid is computed as the average of all points assigned to that cluster:

$$\mu_k = \frac{1}{|C_k|} \sum_{u_i \in C_k} u_i$$

For every pixel representation u_i , the squared Euclidean distance to each centroid is computed:

$$d(u_i, \mu_k) = \|u_i - \mu_k\|^2$$

EStep: Each pixel is assigned to the nearest centroid:

$$\text{Label} (u_i) = \arg \min_k d(u_i, \mu_k)$$


```
def kmeans(features, labels, n_clusters, image_shape, max_iter=100):
    frame_files = []
    first_frame = os.path.join(OUTPUT_DIR, "frame_000.png")
    save_segmentation(labels, image_shape, first_frame)
    frame_files.append(first_frame)
    for iteration in range(max_iter):
        centers = []
        for k in range(n_clusters):
            cluster = features[labels == k]
            if len(cluster) == 0:
                centers.append(np.zeros(features.shape[1], dtype=np.float32))
            else:
                centers.append(np.mean(cluster, axis=0))
        centers = np.array(centers, dtype=np.float32)
        distances = cdist(features, centers, metric="sqeuclidean")
        new_labels = np.argmin(distances, axis=1)
        filename = os.path.join(OUTPUT_DIR, f"frame_{iteration+1:03d}.png")
        save_segmentation(new_labels, image_shape, filename)
        frame_files.append(filename)
        changed = np.sum(labels != new_labels)
        print(f"Iteration {iteration+1}: {changed} pixels changed")
        if np.array_equal(labels, new_labels):
            print("Converged")
            break
        labels = new_labels
    return labels, frame_files
```

→**Saving the Segments and creating Gifs:** This function converts cluster labels into a color image for visualization.

Each cluster is assigned a predefined RGB color.

```
def save_segmentation(labels, image_shape, filename):
    cluster_colors = np.array([[255, 0, 0], [0, 255, 0], [0, 0, 255], [255, 255, 0], [255, 0, 255], [0, 255, 255]])
    rgb = cluster_colors[labels % len(cluster_colors)]
    rgb = rgb.reshape(image_shape[0], image_shape[1], 3)
    img = Image.fromarray(rgb.astype(np.uint8))
    img.save(filename)
```

This create_gif() function visualizes the clustering process by generating an animated GIF.

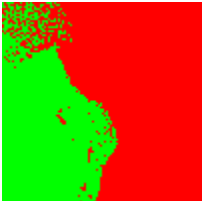
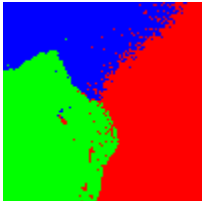
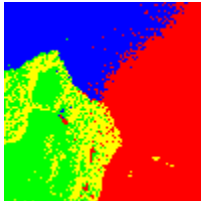
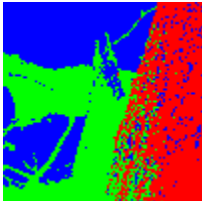
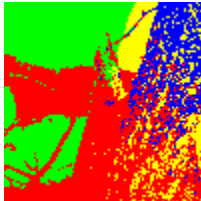
1. Load every segmentation image generated during the iterations.
2. Store the images as frames.
3. Combine the frames into a GIF.
4. The resulting animation shows how cluster assignments evolve until convergence.

```
def create_gif(frame_files, output_name):
    gif_dir = os.path.join(os.path.dirname(output_name), "gif")
    os.makedirs(gif_dir, exist_ok=True)
    output_path = os.path.join(gif_dir, os.path.basename(output_name))
    frames = []
    for file in frame_files:
        img = Image.open(file)
        frames.append(img.copy())
        img.close()
    frames[0].save(output_path, format="GIF", append_images=frames[1:], save_all=True, duration=500, loop=2)
```

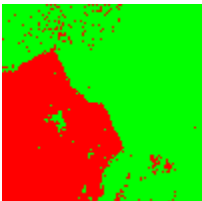
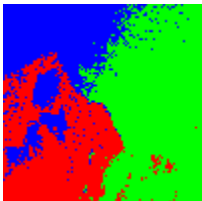
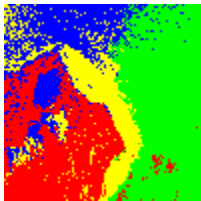
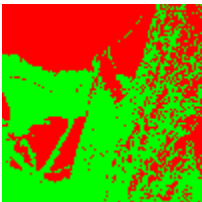
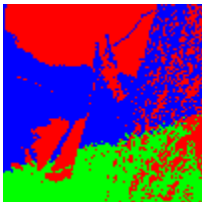
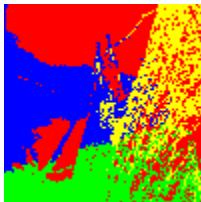
Part II: Results

I. K means Cluster

1. **K-means Clustering: taken (k++ initialisation , $\text{GAMMA_S} = 0.001$, $\text{GAMMA_C} = 0.001$)**

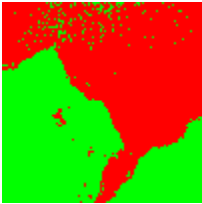
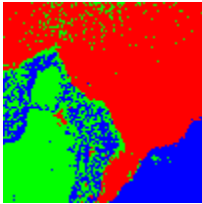
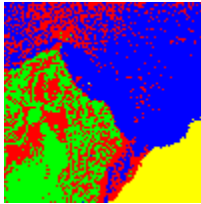
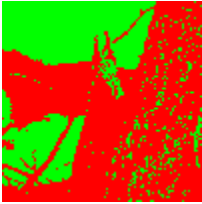
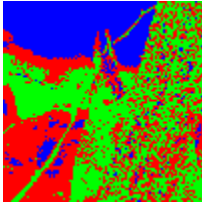
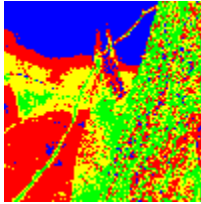
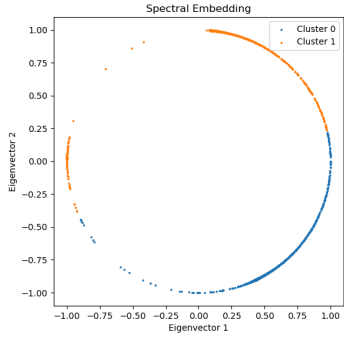
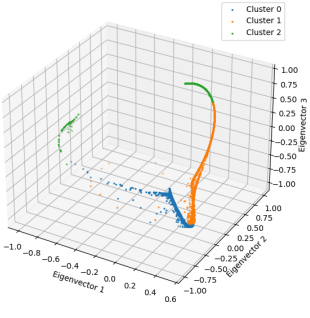
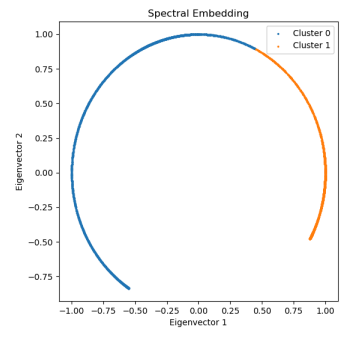
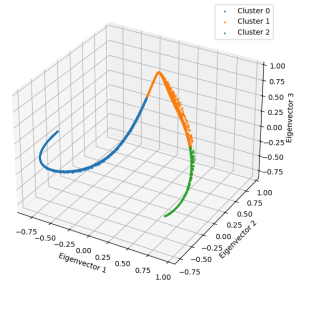
Image	K=2	K=3	K=4
Image 1			
Image 2			

2. **K-means Clustering: taken (Random cluster initialisation, $\text{GAMMA_S} = 0.001$, $\text{GAMMA_C} = 0.001$)**

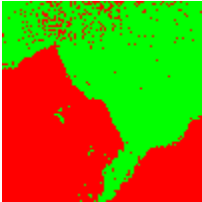
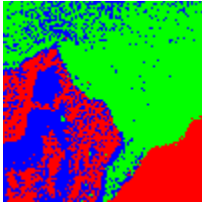
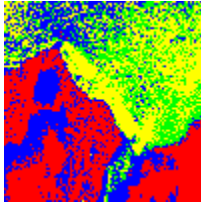
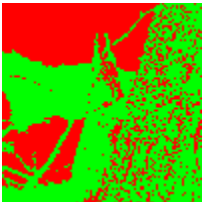
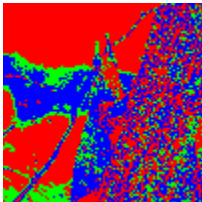
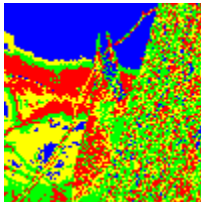
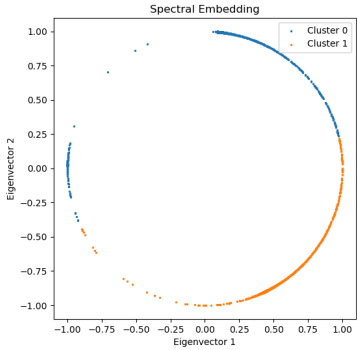
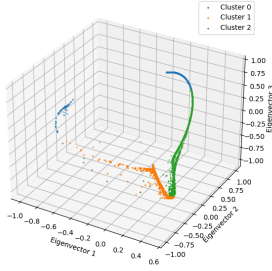
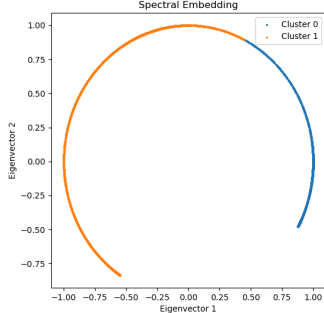
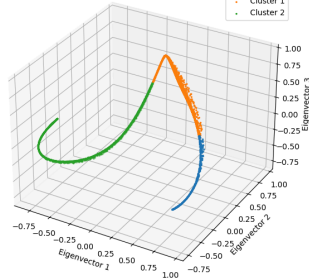
Image	K=2	K=3	K=4
Image 1			
Image 2			

II. Spectral Cluster

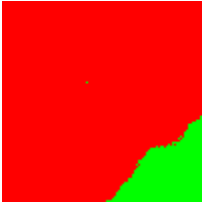
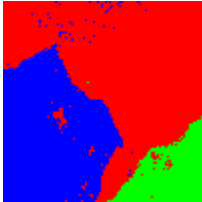
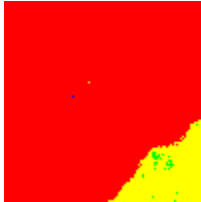
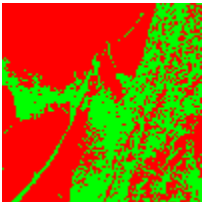
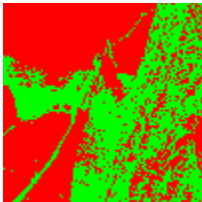
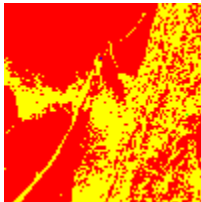
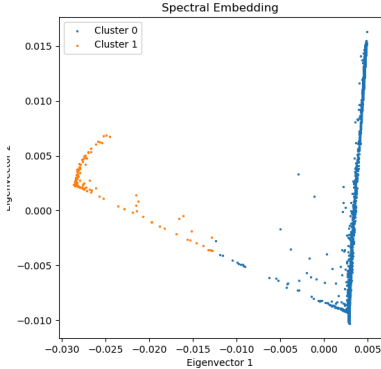
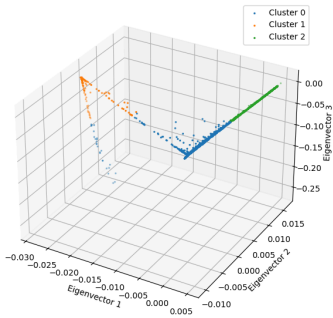
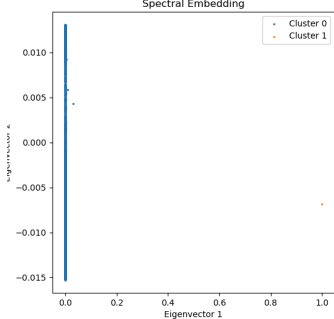
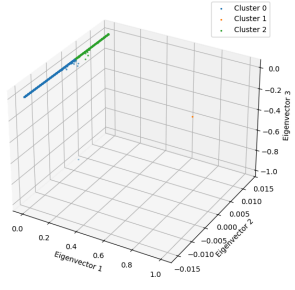
1. Spectral Clustering: taken (Normalized Cut) (k++ initialisation, $\text{GAMMA_S} = 0.0001$, $\text{GAMMA_C} = 0.001$)

Image	K=2	K=3	K=4
Image 1			
Image 2			
 Image 1 : K=2			
 Image 1 : K=3			
 Image 2 : K=2			
 Image 2 : K=3			

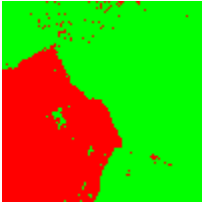
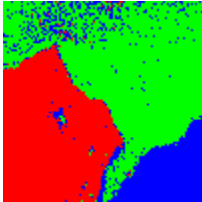
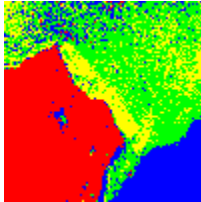
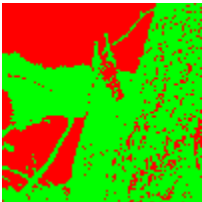
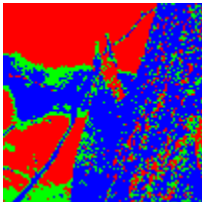
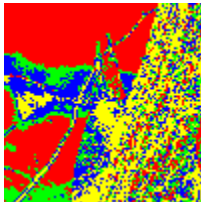
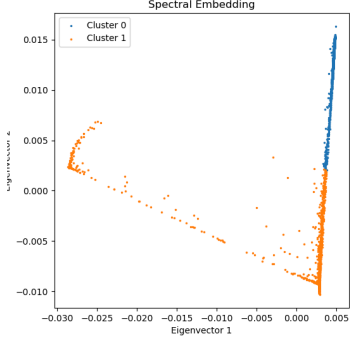
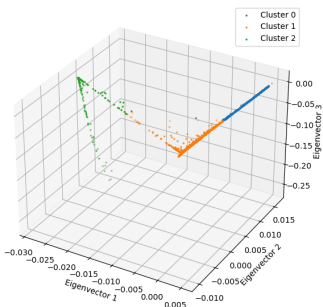
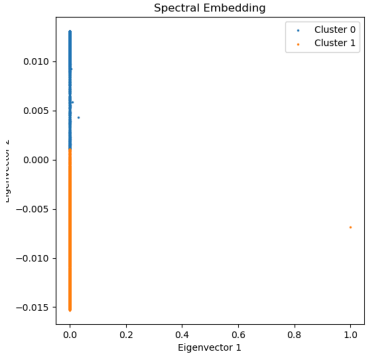
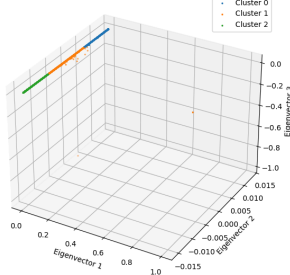
2. Spectral Clustering: taken (Normalized Cut) (random cluster initialisation, $\text{GAMMA_S} = 0.0001$, $\text{GAMMA_C} = 0.001$)

Image	K=2	K=3	K=4
Image 1			
Image 2			
 Image 1 : K=2		 Image 1 : K=3	
 Image 2 : K=2		 Image 2 : K=3	

3. Spectral Clustering: taken (Ratio Cut) (K++ means initialisation, $\text{GAMMA_S} = 0.0001, \text{GAMMA_C} = 0.001$)

Image	K=2	K=3	K=4
Image 1			
Image 2			
 Image 1 : K=2		 Image 1 : K=3	
 Image 2 : K=2		 Image 2 : K=3	

4. Spectral Clustering: taken (Ratio Cut) (random cluster initialisation, $\text{GAMMA_S} = 0.0001$, $\text{GAMMA_C} = 0.001$)

Image	K=2	K=3	K=4
Image 1			
Image 2			
 Image 1: K=2		 Image 1: K=3	
 Image 1: K=2		 Image 1: K=3	

Part III: Conclusions

1. Both Kernel K-Means and Spectral Clustering successfully segmented the images by utilizing spatial and color similarities encoded in kernel matrix.
2. Kernel K-Means directly clusters pixels in the kernel-induced feature space, while Spectral Clustering first transforms the data into a spectral embedding obtained from the graph Laplacian before applying K-Means.
3. K-Means++ initialization generally produced more stable and consistent segmentation results than random initialization because the initial cluster centers were selected to be well separated.
4. Normalized Cut typically generates more balanced clusters than Ratio Cut because the normalized Laplacian reduces the influence of highly connected nodes and cluster size imbalance.
5. Increasing the number of clusters allowed good and better image partitioning.
6. The generated GIF visualizations showed the convergence behavior of both algorithms and how cluster assignments evolved during the iterative optimization process.